

PaviqLabs Assignment

Mini-RAG

"Chat with Your Data"

Full-Stack MERN Implementation Plan with Git Commit Strategy

MongoDB

Express

React + Vite

Node.js

Cohere Embeddings

Gemini LLM

The Golden Rule — Commit After Every Phase

PaviqLabs evaluates your commit history. They want to see HOW you approach a problem step by step — not just the final code. Regular, meaningful commits show your thinking process.

1

One commit per phase minimum

Never finish a phase without committing. Even partial progress counts.

2

Meaningful commit messages

Bad: "update". Good: "feat: add text chunking with 500-char size and 50-char overlap"

3

Never batch everything at the end

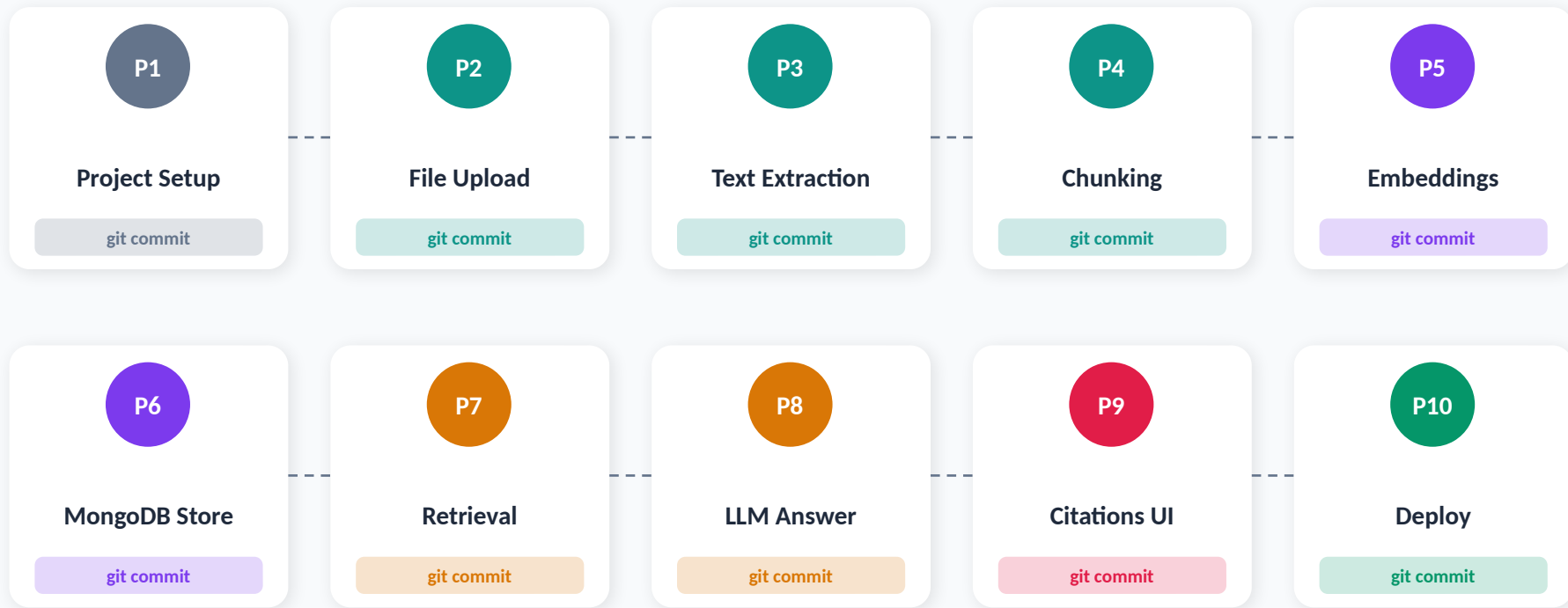
Single-commit repos are instant red flags. Spread over multiple days ideally.

4

Commit working code only

Each commit should be runnable. Don't commit broken states.

10-Phase Loop Map — Build → Commit → Repeat



Each box = one phase. Bottom badge = git commit checkpoint. Color = feature group.

Phases 1-3: Setup, Upload & Text Extraction

P1 Project Setup

~30 min

- Init Git repo + first commit
- Create /backend and /frontend folders
- npm init + install Express, Mongoose, Multer, dotenv
- Create React + Vite frontend
- Setup .env with COHERE_KEY, GEMINI_KEY, MONGO_URI

```
git commit -m "chore: init MERN project  
structure with env config"
```

P2 File Upload Endpoint

~45 min

- Install Multer (npm i multer)
- POST /api/upload route
- Accept PDF or .txt files only
- Save to /uploads folder temporarily
- Return success + file path to frontend

```
git commit -m "feat: add file upload  
endpoint with Multer (PDF + txt)"
```

P3 Text Extraction

~30 min

- Install pdf-parse (npm i pdf-parse)
- Extract raw text from PDF uploads
- For .txt files: plain fs.readFile()
- Clean text (remove extra whitespace)
- Store extracted text in memory for next step

```
git commit -m "feat: text extraction from  
PDF using pdf-parse and .txt"
```

Phases 4-6: Chunking, Embeddings & MongoDB Storage

P4 Text Chunking

~20 min

- Write chunkText(text, size=500, overlap=50)
- Split extracted text into arrays of strings
- Each chunk ~500 chars, 50 char overlap
- Test with sample text manually
- Log chunk count to verify

Tip: Overlap prevents context loss at boundaries

```
git commit -m "feat: chunking logic 500-char  
size, 50 overlap"
```

P5 Cohere Embeddings

~45 min

- npm install cohere-ai
- Setup Cohere client with API key
- Call embed-english-v3.0 model
- Convert each chunk → vector (1024 dims)
- Handle rate limits with small delay

Tip: Never log API key — use process.env only

```
git commit -m "feat: embed chunks via Cohere  
embed-english-v3.0"
```

P6 MongoDB Storage

~40 min

- Create Chunk schema: {text, embedding, docId, index}
- Connect MongoDB Atlas (free 512MB tier)
- Save all chunks after embedding
- Add docId to track which document
- Test: verify chunks in Atlas dashboard

Tip: docId lets you support multiple docs later

```
git commit -m "feat: store chunks +  
embeddings in MongoDB Atlas"
```

Phases 7-9: Retrieval, LLM Answer & Citations UI

P7 Cosine Retrieval

~45 min

- POST /api/chat endpoint
- Embed user question via Cohere
- Load all chunks from MongoDB
- Calculate cosine similarity (JS function)
- Return top-3 most similar chunks

Tip: Sort by score desc, slice top 3

```
git commit -m "feat: cosine similarity  
retrieval, top-3 chunks"
```

P8 Gemini LLM Answer

~40 min

- npm install @google/generative-ai
- Build context string from top-3 chunks
- Prompt: 'Answer ONLY from context below'
- Call Gemini 1.5 Flash API
- Return answer + source chunks in response

Tip: Instruct LLM to say 'not found' if no context

```
git commit -m "feat: LLM answer generation  
via Gemini 1.5 Flash"
```

P9 Citations + React UI

~90 min

- Build Upload panel (drag & drop)
- Build Chat interface (message bubbles)
- Show source citations below each answer
- Highlight which chunk was used
- Responsive layout (Tailwind or CSS)

Tip: Citations are the most valued bonus feature!

```
git commit -m "feat: React UI with upload,  
chat, and citation cards"
```

Phase 10: Deploy + Your Ideal Commit History

P10 Deploy Everything

MongoDB Atlas: Free cluster → paste MONGO_URI in .env

Render.com: Deploy backend → set all env vars in dashboard

Vercel: Deploy React frontend → set VITE_API_URL

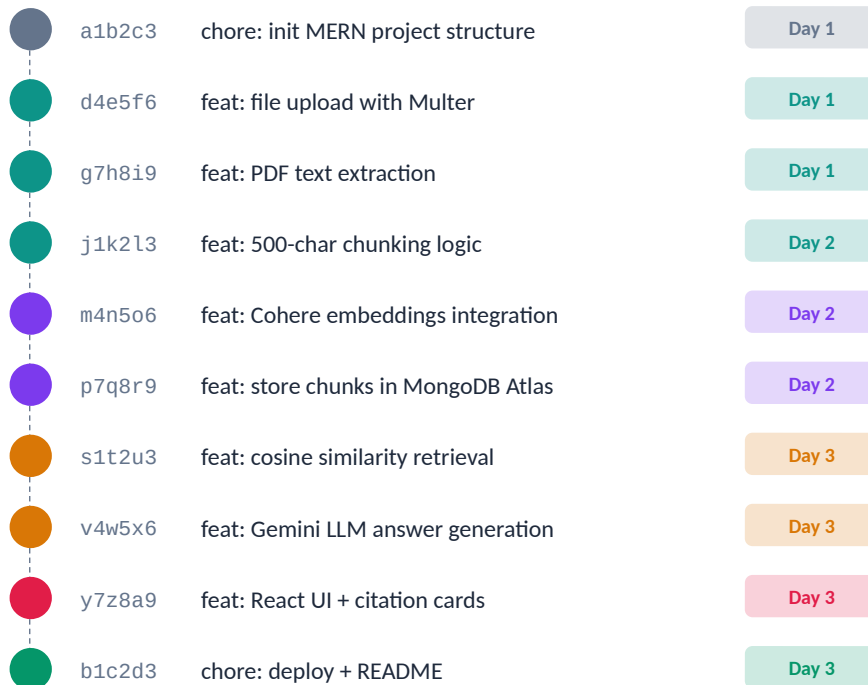
CORS fix: Allow Vercel domain in Express cors() config

README: Write setup steps, tech choices, improvements

Test live URL: Upload PDF, ask a question, verify answer works

```
git commit -m "chore: deploy backend on Render, frontend on Vercel"
```

Ideal Git Commit History



Tech Stack, APIs & Cost Summary

Frontend

React + Vite

Upload UI + chat interface

Backend

Node.js + Express

REST API, file handling, RAG logic

Database

MongoDB Atlas

Stores chunks + embeddings

Embeddings

Cohere embed-v3.0

1024-dim vectors, free tier

LLM

Gemini 1.5 Flash

Answer generation, free tier

File Parse

pdf-parse npm

Extract text from PDF

Upload

Multer npm

Handle multipart/form-data

Deploy BE

Render.com

Free tier backend hosting

Deploy FE

Vercel

Free React hosting

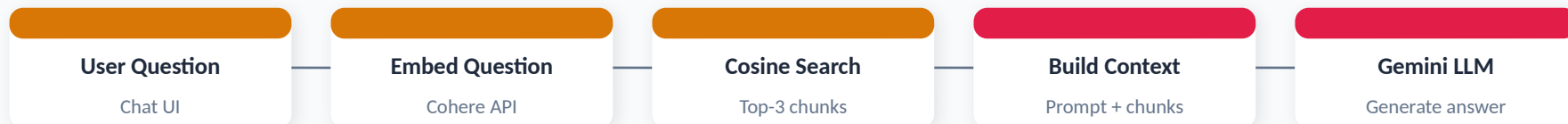
Total Cost = Rs. 0 | All APIs have generous free tiers for this assignment scale

How the RAG Loop Works — End to End

Upload Flow



Query Flow



Answer shown

"Based on the document, the answer is..."

Source chunk 1: "...relevant excerpt from document..." (chunk #3, page 2)

Bonus features implemented

- ✓ Citations (most valued)
- ✓ Graceful no-info response
- ✓ Multiple document support

You're Ready.

Follow the loop. Commit after each phase. Ship by 2 July.

10

Phases to build

10+

Git commits to make

Rs.0

Total API cost

3

Days deadline

- Build a little → commit → test → repeat
- Never push everything in one commit
- Citations bonus is most valued — build it
- README must explain chunking approach clearly

Good luck Arpit — Cosmovexa Coding!